

Régression gaussienne : du modèle linéaire bayésien du cours à une implémentation avec `scikit-learn` et à la main

1 Introduction

L'objectif de ce travail est de comprendre, puis de reproduire, une régression gaussienne à partir du cadre présenté dans le polycopié. Le point de départ est un modèle *linéaire bayésien* dans lequel les coefficients sont gaussiens, puis nous ajouterons un paramètre au modèle, les observations sont alors obtenues en ajoutant un bruit gaussien. En pratique, l'exploration avec `scikit-learn` montre rapidement que plusieurs questions supplémentaires apparaissent : quel noyau est réellement utilisé, comment introduire un biais, comment interpréter les écarts-types prédits, et comment estimer les hyperparamètres du modèle.

2 Le modèle linéaire bayésien du cours

Dans le cours, on considère le modèle

$$Y = xW + N, \quad (1)$$

où $x \in \mathbb{R}^{n \times p}$ est la matrice de design, $W \in \mathbb{R}^p$ est un vecteur aléatoire gaussien et $N \in \mathbb{R}^n$ un bruit gaussien indépendant. Les hypothèses sont

$$W \sim \mathcal{N}(0, \Gamma_W), \quad N \sim \mathcal{N}(0, \sigma^2 I_n), \quad W \perp N. \quad (2)$$

On en déduit que le vecteur des observations suit la loi

$$Y \sim \mathcal{N}(0, X\Gamma_W X^\top + \sigma^2 I_n). \quad (3)$$

Dans le cas gaussien, l'inférence a posteriori est explicite. Une fois $Y = y$ observé, la loi de $W \mid Y = y$ reste gaussienne, avec une moyenne et une covariance obtenues par conditionnement gaussien. Pour une nouvelle entrée $x_0 \in \mathbb{R}^p$, si l'on définit

$$Y_0 = x_0^\top W + N_0, \quad N_0 \sim \mathcal{N}(0, \sigma^2), \quad (4)$$

alors on obtient aussi directement l'espérance et la variance conditionnelles de $Y_0 \mid Y = y$.

Dans ce cadre, la fonction latente est linéaire en x .

3 Utilisation de `scikit-learn` : le rôle du noyau

Nous avons d'abord utilisé un `GaussianProcessRegressor` de `scikit-learn` sur des données synthétiques générées à partir de la fonction non linéaire

$$f(x) = x \sin x. \quad (5)$$

Avec le modèle par défaut, on voit sur 1 que la prédiction suit correctement la non-linéarité de la fonction. Cela peut donner l'impression qu'une régression gaussienne sait naturellement produire des comportements non linéaires.

En réalité, ce comportement vient du noyau utilisé. Dans `scikit-learn`, si aucun noyau n'est précisé, le modèle emploie par défaut un noyau de type RBF, qui définit une covariance bien plus riche que le produit scalaire canonique. Cette première étape a donc mis en évidence un point important : pour comprendre ce que fait le logiciel, il faut distinguer la théorie du cours et le choix effectif du noyau dans l'implémentation.

Lorsque l'on impose explicitement un noyau linéaire `DotProduct`, on voit sur 2 que le comportement change complètement : la prédiction redevient affine, et elle ne peut plus approcher convenablement une fonction non linéaire comme $x \sin x$. Cette comparaison a servi de point de départ à toute la suite : le cours traite du cas linéaire, mais il semble usuel de changer le produit scalaire.

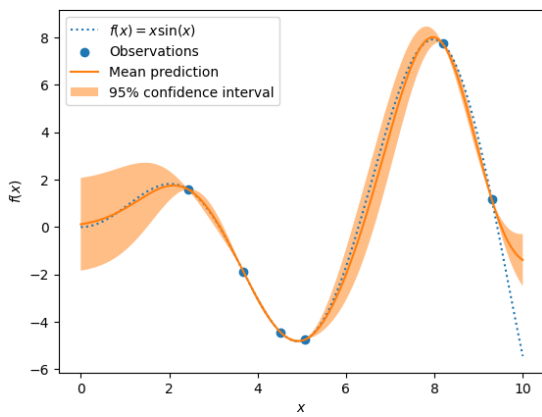


FIGURE 1 – Régression avec le kernel par défaut

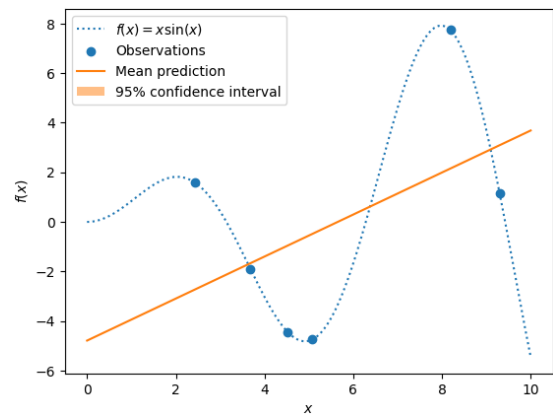


FIGURE 2 – Régression en imposant un kernel affine

4 Lecture du modèle linéaire comme noyau gaussien

Le lien entre le modèle du cours et la formulation par noyau apparaît en réécrivant la covariance des observations. Si l'on fixe pour $x, x' \in \mathbb{R}^p$

$$k(x, x') = x^\top \Gamma_W x', \quad (6)$$

alors, pour des données d'entraînement $x_1, \dots, x_n$, on note

$$K = (k(x_i, x_j))_{1 \leq i, j \leq n} = X \Gamma_W X^\top. \quad (7)$$

La covariance de Y devient alors

$$\text{Cov}(Y) = K + \sigma^2 I_n. \quad (8)$$

Le passage au formalisme « noyau » ne change donc pas le modèle ; il donne simplement une autre manière de l'écrire.

Cette reformulation est importante pour deux raisons. D'abord, elle permet de comprendre directement le comportement de `scikit-learn` lorsqu'on choisit `DotProduct`. Ensuite, elle ouvre la voie à des noyaux plus généraux : remplacer $k(x, x') = x^\top \Gamma_W x'$ par un autre noyau revient à changer la famille de fonctions autorisées par le modèle.

5 Ajout d'un biais : du modèle linéaire au modèle affine

Le modèle du cours ne contient pas explicitement de biais. Or, sur les données réelles, qu'on a bien sûr centré-réduit, le terme en $x = 0$ n'a pas de raison d'être nul. Cela est d'ailleurs confirmé par l'estimation obtenue avec `scikit-learn`. Il est donc naturel de considérer le modèle affine

$$Y = xW + b\mathbf{1} + N, \quad (9)$$

où b est un scalaire aléatoire indépendant de W et de N , avec

$$b \sim \mathcal{N}(0, \sigma_0^2). \quad (10)$$

On doit donc refaire les calculs du cours. Pour cela, on introduit

$$\widetilde{W} = \begin{pmatrix} W \\ b \end{pmatrix}, \quad \widetilde{x} = \begin{pmatrix} x & \mathbf{1} \end{pmatrix}, \quad \widetilde{\Gamma} = \begin{pmatrix} \Gamma_W & 0 \\ 0 & \sigma_0^2 \end{pmatrix}. \quad (11)$$

Alors le modèle s'écrit simplement

$$Y = \widetilde{x}\widetilde{W} + N. \quad (12)$$

Toutes les formules du cours restent valables mot pour mot en remplaçant W, X, Γ_W par leurs versions tilde et en faisant les calculs par blocs. Cela prouve rigoureusement que l'ajout d'un biais est une généralisation directe du cadre initial.

On peut aussi faire le lien avec la notation du noyau. En effet, en développant la covariance, on obtient

$$\text{Cov}(Y) = x\Gamma_W x^\top + \sigma_0^2 \mathbf{1}\mathbf{1}^\top + \sigma^2 I_n. \quad (13)$$

Le biais aléatoire se traduit donc par l'ajout d'un terme constant dans le noyau :

$$k(x, x') = x^\top \Gamma_W x' + \sigma_0^2. \quad (14)$$

Dans `scikit-learn`, c'est le rôle du paramètre `sigma_0` du noyau `DotProduct`. On retrouve ainsi l'interprétation probabiliste de ce paramètre : il contrôle la variance a priori du biais global.

6 Données synthétiques bruitées : validation du cadre linéaire

Après les tests sur la fonction non linéaire $x \sin x$, on revient donc à un exemple compatible avec les hypothèses du cours : une relation linéaire bruitée. Dans ce cas, l'utilisation d'un noyau `DotProduct` sans biais, et d'un bruit gaussien connu, produit des prédictions cohérentes (voir 3). Ceci valide le fait que le noyau linéaire n'est pas un mauvais choix en soi ; il devient inadéquat seulement lorsque la vraie relation est non linéaire.

Elle a également permis de clarifier le rôle du bruit dans `scikit-learn`. Il faut utiliser un `WhiteKernel` en plus dans le modèle, ce qui n'est pas fait par défaut.

7 Passage aux données réelles : le jeu *Airfoil*

L'étude est ensuite menée sur le jeu de données *Airfoil Self-Noise*. Une étape préliminaire de standardisation est indispensable, car le noyau linéaire dépend directement des produits scalaires entre vecteurs d'entrée. Sans normalisation, des variables portées à des échelles très différentes domineraient artificiellement la covariance. Par exemple la fréquence en Hertz vaut plusieurs

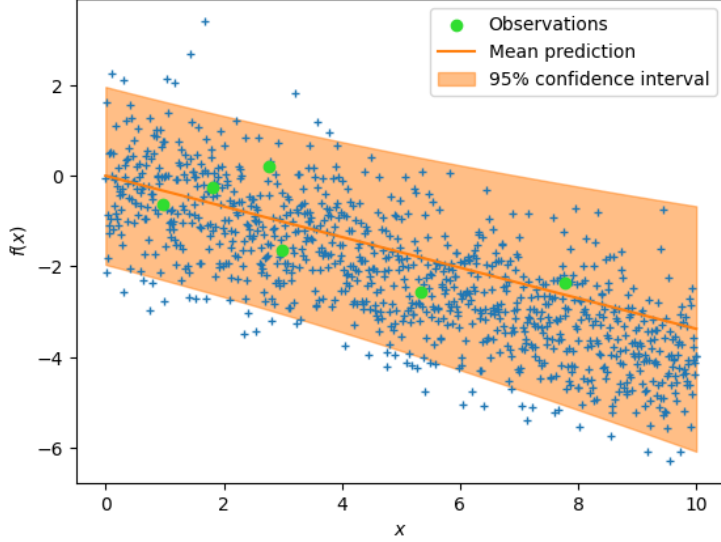


FIGURE 3 – Régression gaussienne avec un modèle linéaire

milliers, alors que d'autres grandeurs sont de l'ordre de l'unité. La transformation utilisée sur les données de test doit ensuite être exactement celle apprise sur les données d'entraînement.

Sur ce jeu de données, nous verrons que le noyau linéaire avec biais apparaît raisonnable : il donne des performances correctes, et l'hyperparamètre associé au biais n'est pas négligeable. Cela justifie le passage d'un modèle strictement linéaire à un modèle affine.

8 Estimation des hyperparamètres par maximum de vraisemblance marginale

Une fois le modèle fixé, il reste à choisir ses hyperparamètres. Dans l'implémentation maison, on a choisi de garder

$$\Gamma_W = I_p, \quad (15)$$

ce qui revient à prendre un noyau linéaire isotrope, puis d'estimer les deux hyperparamètres

$$\theta = (\sigma_0, \sigma). \quad (16)$$

Principe

Le point clé est que l'on n'optimise pas la variable W elle-même. On part de la covariance

$$K_\theta = XX^\top + \sigma_0^2 \mathbf{1}\mathbf{1}^\top + \sigma^2 I_n, \quad (17)$$

puis on considère la densité gaussienne

$$p(y | X, \theta) = \mathcal{N}(0, K_\theta)(y). \quad (18)$$

La log-vraisemblance vaut donc

$$\log p(y | X, \theta) = -\frac{1}{2}y^\top K_\theta^{-1}y - \frac{1}{2}\log \det(K_\theta) - \frac{n}{2}\log(2\pi). \quad (19)$$

On choisit alors

$$\hat{\theta} = \arg \max_{\theta} \log p(y | X, \theta). \quad (20)$$

Idéalement, dans une approche bayésienne complète, il faudrait aussi mettre un prior sur les hyperparamètres et intégrer sur leur loi a posteriori. En pratique, cette stratégie est difficile à mettre en place, c'est pour cela qu'on fixe σ_0 . C'est une approximation : on choisit un hyperparamètre optimisant la vraisemblance marginale, puis on effectue toute l'inférence conditionnellement à ce choix. Cette approximation est celle effectuée par `scikit-learn`.

Conceptuellement, cela signifie que l'on distingue deux niveaux :

- un niveau *modèle*, dans lequel on calibre σ_0 et σ à partir des données observées ;
- un niveau *inférence*, dans lequel on calcule la loi a posteriori des variables latentes et la distribution prédictive en supposant ces hyperparamètres fixés.

9 Implémentation numérique

On a donc tous les éléments pour effectuer la régression. Nous avons commencé par tester la version de `scikit-learn`, mais nous essayons aussi de l'implémenter nous-même. Les résultats trouvés (dont les valeurs des hyperparamètres) sont sensiblement les mêmes.

La tentative d'implémentation naïve du maximum de vraisemblance a échoué pour des raisons numériques : calcul direct de K^{-1} , calcul de $\det(K)$, et optimisation sur des paramètres contraints à être positifs. Ces difficultés ont été progressivement levées.

La version finale repose sur trois choix numériques usuels pour ce genre de problèmes.

1) Optimisation en paramètres logarithmiques. On optimise $\log \sigma_0$ et $\log \sigma$ plutôt que σ_0 et σ eux-mêmes. Cela garantit automatiquement la positivité et améliore la stabilité de l'optimisation.

2) Décomposition de Cholesky. Au lieu de calculer explicitement K^{-1} et $\det(K)$, on factorise

$$K = LL^\top, \quad (21)$$

avec L triangulaire inférieure, via la décomposition de Cholesky. Alors

$$\log \det(K) = 2 \sum_{i=1}^n \log L_{ii}, \quad (22)$$

et le terme avec l'inverse de K est calculé en résolvant un système linéaire.

3) Ajout d'un epsilon. On remplace K par $K + \varepsilon I_n$ avec ε très petit, pour éviter les problèmes numériques de singularité.

Une fois les hyperparamètres estimés, la prédiction sur un nouveau point x_* se fait à l'aide des quantités

$$k_*(x_*) = Xx_* + \sigma_0^2 \mathbf{1}, \quad k_{**}(x_*) = x_*^\top x_* + \sigma_0^2 + \sigma^2, \quad (23)$$

et des formules qu'on peut retrouver à l'aide de ce qui a été fait plus haut

$$\mathbb{E}[Y_* | Y = y] = k_*^\top K^{-1} y, \quad (24)$$

$$\text{Var}(Y_* | Y = y) = k_{**} - k_*^\top K^{-1} k_*. \quad (25)$$

Lorsque l'on prédit plusieurs points simultanément, la variance prédictive devient naturellement une matrice de covariance complète, donc on récupère d'abord une matrice carrée dont on extrait la diagonale.

10 Résultats

On peut commencer par regarder l'utilisation des différents noyaux avec `scikit-learn`, confirmant notre choix de modèle :

Modèle	Mean log-likelihood	Remarques
Kernel par défaut (RBF)	-6.64×10^9	Sans bruit
Kernel RBF + bruit	-6.24	Amélioration drastique
Produit scalaire + bruit	-2.97	Meilleur modèle

TABLE 1 – Comparaison des modèles de régression gaussienne en termes de log-vraisemblance prédictive moyenne.

Ensuite, l'implémentation maison a finalement donné des résultats presque identiques à ceux de `scikit-learn`, à la fois sur la moyenne prédite, sur la variance prédictive marginale et sur les métriques d'évaluation probabiliste. En particulier, la couverture empirique des intervalles à 95% est très proche de la valeur attendue, ce qui indique que l'incertitude prédite est correctement calibrée. La log-vraisemblance donne un valeur très raisonnable, la régression gaussien linéaire est donc très satisfaisante. On peut visualiser avec une PCA, en une dimension, les résultats de la régression sur la figure 4.

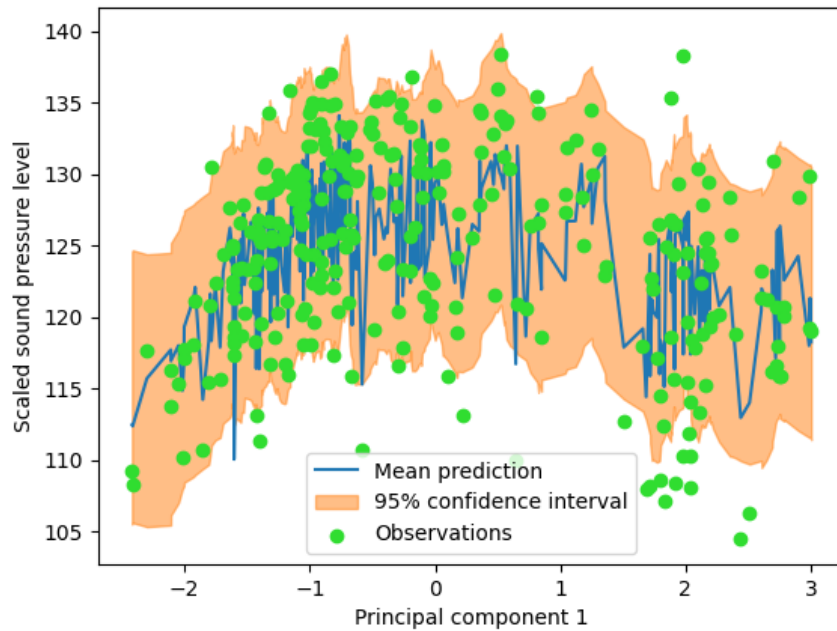


FIGURE 4 – Représentation de de la régression gaussienne via PCA